

EX 1(a): Text Generation using N-gram Language Model

Python Program

```
!pip install nltk

import nltk
from nltk.corpus import brown
from collections import Counter, defaultdict

nltk.download("brown")
nltk.download("universal_tagset")

sentences = brown.sents()
tokens = [word.lower() for sentence in sentences for word in sentence]

def build_ngram_model(tokens, n):
    model = defaultdict(Counter)
    for i in range(len(tokens) - n):
        context = tuple(tokens[i:i+n-1])
        next_word = tokens[i+n-1]
        model[context][next_word] += 1
    return model

bigram_model = build_ngram_model(tokens, 2)
trigram_model = build_ngram_model(tokens, 3)

def predict_next_word(model, context, top_k=5):
    context = tuple(context)
    if context not in model:
        return []
    return model[context].most_common(top_k)

print(predict_next_word(bigram_model, ["the"]))
print(predict_next_word(trigram_model, ["in", "the"]))
```

Sample Output

```
Total sentences: 57340
Total tokens: 1161192

Bigram Prediction for 'the':
same, first, new, other, most

Trigram Prediction for ['in', 'the']:
case, same, state, city, country
```